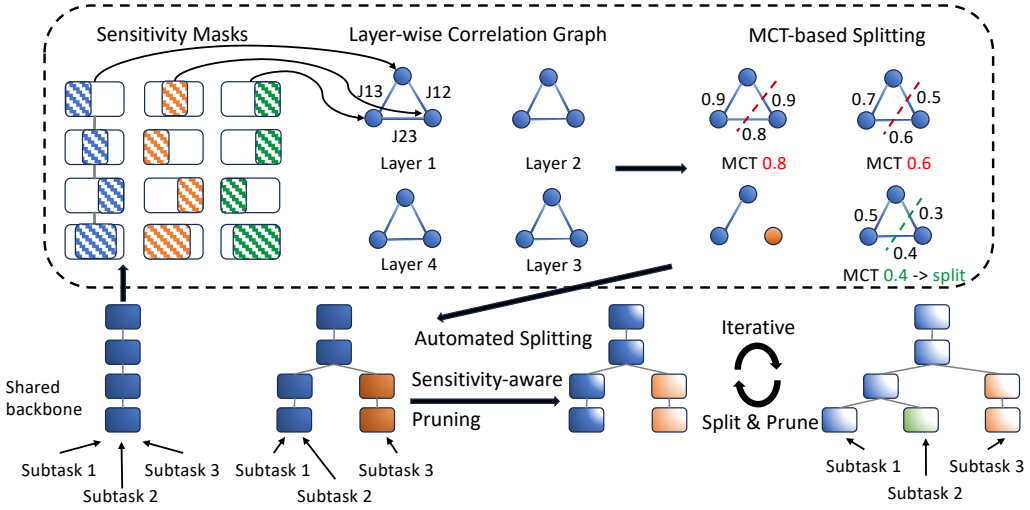

Figure 3. **Iterative splitting and pruning.** Starting from a shared backbone, we compute the layer-wise sensitivity mask $\mathcal{M}$ for each subtask loss, and calculate pair-wise IoU score J across different subtasks for the layer-wise correlation graph. Model is split at the layer with a small minimal cutting threshold (MCT), and, then, is pruned globally. Applying splitting and pruning in an iterative fashion leads to the final Split-Ensemble architecture that satisfies computational cost constraints.

proper training objectives to improve both accuracy and uncertainty calibration for submodels learning these subtasks in Section 3.2, and how final classification and OOD detection are performed with the ensemble of subtask models in Section 3.3, as in Figure 2.

3.1. Complementary Subtask Splitting

Consider a classification task with N classes in total. Here we define the process of complementary subtask splitting as grouping the classes into n groups with K_i classes in the i-th group, where $\sum_{i=1}^{n} K_i = N$, and each class is only contained in one group. This splits the original classification task into n subtasks with the i-th class considering only the K_i classes as in-distribution (ID), while all the other classes are out-of-distribution (OOD) for this subtask.

For training, it is possible to train a submodel for each subtask separately with only the data within the K_i classes. However, at inference, we do not know in advance which submodel we should assign an input with the unknown class to. Therefore, each submodel still needs to handle images from all the classes, not only within the K_i classes. To address that, we add an additional class, namely "OOD" class, to the subtask to make it a (K_i+1)-way classification task. In the training of the corresponding submodel, we use the entire dataset of the original task with a label conversion. For images in the K_i ID classes of the subtask, we assign them label 0 through $K_i - 1$ in the subtask. For all the other images from the $N - K_i$ OOD classes, we assign the same label K_i to them indicating the OOD class. In this way, a well-trained submodel classifier on the subtask can correctly classify the images within its ID classes, and reject other classes as OOD. Then each input image from the original task will be correctly classified by one and only one submodel, while being rejected by all others.

The grouping strategy for the subtask splitting will affect the final performance. To enable each submodel to learn a generalizable distinction between the ID and OOD classes more easily, we intend to have the ID classes semantically closer to each other while away from the OOD classes. In most large-scale real-world datasets, it is by nature that some classes are semantically closer than others, and all classes can be grouped into multiple semantically close groups (e.g. the superclass in CIFAR-100 (Krizhevsky & Hinton, 2009) or the WordNet hierarchy in ImageNet (Deng et al., 2009)). In the case where the semantically similar groups are not naturally available (e.g. in CIFAR-10), we cluster the classes based on mean hidden layer features of each class's training data on a pretrained classifier of the original task, where classes with closer features are grouped together. We empirically verify our intuition on the semantically close grouping in Table 7 in the Appendix.

3.2. Submodel Training Objective

Here we derive the training objective for the submodel to learn generalizable uncertainty estimation on each subtask. Without loss of generality, we consider a subtask with K in-distribution classes out of the total N classes in the derivation. The existence of OOD classes in each subtask enables us to perform OOD-aware training for each submodel, without utilizing external data outside of the original task. Hence,

we take inspiration from outlier exposure (Hendrycks et al., 2018), where we train with normal one-hot label for ID data, but use a uniform label for OOD data to prevent the model from over-confidence. Formally, for an input from OOD classes, we set the i-th element of the label $\hat{y}^{OOD}$ as

$$\hat{y}_i^{OOD} = \begin{cases} 1/N & 0 \leq i \leq K-1 \\ (N-K)/N & i = K \end{cases}, \quad (1)$$

where class K corresponds to the OOD class. Note that we set the subtask target of ID classes to $1/N$ instead of $1/K$ to make the ID logits comparable across all submodels with different amounts of classes when facing an OOD input. The OOD logit at class K is therefore used as a "sink" to assign the remaining softmax probability. Optimally, all submodels will output a low max probability of $1/N$ when facing an OOD input of the original task, which is as desired for the outlier exposure objective (Hendrycks et al., 2018).

Under the setting of our subtask, each ID class only consists the data from one class in the original task, yet the OOD class corresponds to all data in the rest $N-K$ classes, leading to a $(N-K)\times$ training data to other classes. Directly training with the imbalanced class will lead to significant bias over the OOD class. We refer to recent advances in imbalanced training and use the class-balance reweighing (Cui et al., 2019) as the loss of each class. The weight w_i of class i of the subtask is formulated as

$$w_i = \begin{cases} \frac{1-\beta}{1-\beta^n} & 0 \leq i \leq K-1 \\ \frac{1-\beta}{1-\beta^{(N-K)n}} & i = K \end{cases}, \quad (2)$$

where n is the amount of data in each class of the original task, and $\beta \in [0, 1)$ is a hyperparameter balancing the weight. We follow Cui et al. (2019) to set $\beta = 0.9999$, where ablation study shows the choice of β does not affect the final performance metric. We apply the reweighing on the binary cross entropy (BCE) loss to formulate the submodel training objective $\mathcal{L}_{CB}(X, Y)$ with submodel output logits X and label Y as

$$\mathcal{L}_{CB} = \mathbb{E}_X \sum_{i=0}^{K} \Big[-w_i \big(\hat{y}_i \log \sigma(x_i) + (1-\hat{y}_i) \log(1-\sigma(x_i)) \big) \Big], \quad (3)$$

where $\sigma(\cdot)$ denotes the Sigmoid function, x_i is the i-th element of a output logit x, and $\hat{y}_i$ follows Equation (1). Note that although the submodel learns a multi-class classification as illustrated in Figure 2, we use the summation of BCE loss of each individual class from 0 through k as the submodel training objective. This choice of assuming each category is classified independently has been shown to have advantages in learning from imbalanced datasets (Cui et al., 2019). It also gives us more flexibility in controlling the target labels and class-wise reweighting.

3.3. Ensemble Training and Inference

To get the final output logits for the original task, we concatenate the ID class logits from each submodel into a N-dimensional vector. Then the classification can be performed with an argmax of the concatenated logits. In order to calibrate the range of logits across all submodels, we perform a joint training of all submodels, with the objective

$$\mathcal{L}_{ens} = \sum_i \mathcal{L}_{CB}^i(X_i, \hat{Y}_i) + \lambda \mathcal{L}_{CE}(X, Y). \quad (4)$$

Here X_i and $\hat{Y}_i$ denote the output logits and the subtask target of submodel i, as formulated in Section 3.2. X denotes the concatenated ID logits, Y denotes the label of the original task, and $\mathcal{L}_{CE}$ is the cross entropy loss. Hyperparameter λ balances the losses. Empirically, we find that a small λ (e.g. 1e-4) is enough for the logits ranges to calibrate across submodels, while not driving the ID logits of each submodel to be overconfident.

For uncertainty estimation, we compute the probability of the ensemble model outputting a label y from the contribution of submodel f_i given an input z as $p(y|z) = p(y|z, f_i) \times p(f_i|z)$. Here, $p(y|z, f_i)$ can be estimated with the softmax probability of the y class at the output of submodel f_i. $p(f_i|z)$ can be estimated with $1-$ the softmax probability of the OOD class of submodel f_i, as the probability that f_i provides a valid ID output for the ensemble. With the design of the OOD-aware training objective in Equation (1), we use $p(y|z)$ as the OOD detection criteria. Specifically, a single threshold will be selected so that all input with a smaller probability than the threshold will be considered as OOD sample.

4. Architecture with Splitting and Pruning

In this section, we propose the Split-Ensemble architecture to effectively and efficiently learn the aforementioned subtask-splitting training task. Starting from a single backbone, we discuss the process of submodel splitting in Section 4.1 and formulate the criteria to prune unimportant structures in Section 4.2. Figure 3 overviews our pipeline.

4.1. Correlation-based Automated Splitting

Intuitively, with the subtasks split from the original task, submodels should be able to share the low-level features learned in the early layers of the model, while using diverse high-level filters in later layers in each subtask. Given the difficulty of merging independently training submodels during training, we instead start our ensemble training from a single backbone model for the original task. We consider all the submodels using the same architecture with the same parameters of the backbone model, only with independent fully-connected (FC) classifiers at the end.

The question therefore becomes: **On which layer shall we split a submodel from the shared backbone?** Here we propose an automated splitting strategy that can learn a proper splitting architecture given the split subtasks. Intuitively, two submodels can share the same layer if both of their tasks benefit from the existing weights (i.e., removing a filter will hurt both subtasks simultaneously). Otherwise, if the weight sensitive to subtask 1 can be removed for subtask 2 without harm, and vice versa, then it would be worth splitting the architecture and shrinking the submodels separately. Formally, for submodels sharing the same layer with weight W, we perform a single-shot sensitivity estimation of all the weight elements w_j in W on the loss $\mathcal{L}^i_{CB}$ of each subtask i respectively as

$$s^i_j = \frac{|g(w_j)|}{\sum_{w_k \in W} |g(w_k)|},\ g(w_j) = w_j \nabla_{w_j} \mathcal{L}^i_{CB}(W), \quad (5)$$

following the criteria proposed in SNIP (Lee et al., 2019). Then we perform a Top-K masking to select the K weight elements with the largest sensitivity, forming a sensitive mask $\mathcal{M}_i$ for submodel i. The weight element lies in the intersection of the two masks $\mathcal{M}_i \cap \mathcal{M}_j$ are sensitive for both subtasks, while other elements in the union $\mathcal{M}_i \cup \mathcal{M}_j$ but not in the intersection is only sensitive to one subtask. We use the Intersection over Union (IoU) score to measure the pair-wise mask correlation as $J_{ij} = \frac{|\mathcal{M}_i \cap \mathcal{M}_j|}{|\mathcal{M}_i \cup \mathcal{M}_j|}$, where $|\cdot|$ denotes the cardinality of a set. It has been observed in previous multi-task pruning work that pruning mask correlations will be high in the early layers and drop sharply towards later layers (Sun et al., 2022), as later layers learn more high-level features that are diverse across subtasks. A pair of submodels can be split at the earliest layer where the IoU score drops below a pre-defined threshold. The architecture and parameters of the new branch are initialized as the exact copy of the original layers it is splitting from, which guarantees the same model functionality before and after the split. The branches will then be updated and pruned independently according to their subtask objectives after the splitting is performed.

In the case of multiple submodels, we compute the pair-wise IoU score between each pair of submodels and build a "*correlation graph*" for each layer. The correlation graph is constructed as a weighted complete graph $C = (V, E)$ with each submodel being a node $v \in V$, and the IoU score between two submodels J_{uv} is assigned as the weight of the edge $(u, v) \in E$ between the corresponding nodes. Then a split of the model is equivalent to performing a cut on the correlation graph to form two separated subgraphs S and T. Here we propose a measurement of "*Minimal Cutting Threshold*" (MCT), which is the minimal correlation threshold for edge removal that can cut the correlation graph into two. Formally, the MCT of a correlation graph C is defined as

$$MCT(C) = \min_{S,T} \left[\max_{u \in S, v \in T, (u,v) \in E} J_{uv} \right],\ s.t.\ S + T = V. \quad (6)$$

A small MCT indicates that a group of submodels has a weak correlation with the rest, therefore they can be separated from the shared architecture. In practice, we will iteratively split the earliest layer with an MCT lower than a predefined value in a branch shared by multiple submodels, until all submodels have individual branches or the training ends. The splitting strategy will turn a single backbone model into a tree-like architecture, as illustrated in Figure 3.

4.2. Sensitivity-aware Global Pruning

To remove the redundancy in the submodels for simpler subtasks, we perform global structural pruning on the Split-Ensemble architecture. We perform structural sensitivity estimation on a group of weight element w_s belonging to a structure $\mathcal{S}$ for the loss $\mathcal{L}^i_{CB}$ of each subtask i. We utilize the Hessian importance estimation (Yang et al., 2023), which is computed as

$$\mathcal{I}^i(\mathcal{S}) = \left(\sum_{s \in \mathcal{S}} w_s \nabla_{w_s} \mathcal{L}^i_{CB}(w_s) \right)^2. \quad (7)$$

It has been shown that this importance score is comparable across different layers and different types of structural components (Yang et al., 2023), making it a good candidate for global pruning. Then, we greedily prune a fixed number of filters with the smallest $\mathcal{I}^i$ in submodel i. In the case where multiple submodels are sharing the structure, we separately rank the importance of this structure in each submodel, and will only prune the filters that are prunable for all the submodels sharing it.

Putting everything together, we iteratively perform the aforementioned automated splitting and pruning process during the training of the Split-Ensemble model. Splitting and pruning are performed alternatively. Removing commonly unimportant structures will reduce the sensitivity correlation in the remaining parameters, enabling further splitting of the submodels. In the meantime, having a new split enables additional model capacity for further pruning. The splitting will be fixed when all submodels have an individual branch towards later layers of the model. Pruning will be stopped when the Floating-point Operations (FLOPs) of the Split-Ensemble architecture meet a predefined computational budget, typically the FLOPs of the original backbone model. The Split-Ensemble model will then train with the fixed model architecture for the remaining training epochs. The detailed process of Split-Ensemble training is provided in the pseudo-code in Algorithm 1 of Appendix B.